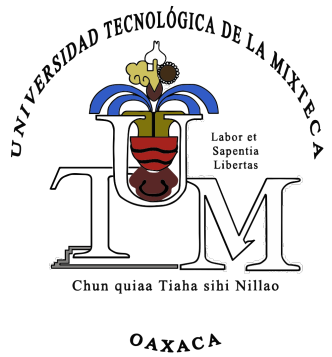


Universidad Tecnológica de la Mixteca

Analizador Sintáctico

Implementación del algoritmo de la Colección canónica

CompiladorEq3 — Asignación 4

Curso: Compiladores
Profesor: David Martínez Torres
Programa: Ingeniería en Computación, quinto semestre

Equipo 3

Arenas Rojas Ivri Nitzani
Bautista Jiménez Israel
Hernández Torija Itzel Amairani
Juárez Jiménez Yarelli Sinai
López López Xochitl Nerina
Quiroz Martínez Maybelin

Oaxaca, México

Índice

Introducción **2**

1. Análisis **2**
1.1. Análisis sintáctico ascendente y gramática aumentada . . . 2
1.2. Elementos LR(0), cerradura e Ir_a . . . 3
1.3. Herramientas utilizadas . . . 3
1.4. Formato del archivo de la gramática aumentada . . . 3

2. Algoritmos de la colección canónica **4**
2.1. Algoritmo de cerradura . . . 4
2.2. Algoritmo Ir_a . . . 4
2.3. Algoritmo de la colección canónica . . . 5
2.4. Terminación del algoritmo . . . 5
2.5. Ejemplo paso a paso: Gramática 1 . . . 5

3. Descripción de la implementación **6**

4. Implementación en Python **7**
4.1. Arquitectura por capas y paquetes . . . 7
4.2. Representación de los elementos y los estados . . . 8
4.3. Gramática aumentada . . . 8
4.4. Cerradura . . . 9
4.5. Ir_a . . . 9
4.6. Colección canónica . . . 9
4.7. Manejo de errores en la gramática . . . 10
4.8. Relación con las entregas anteriores . . . 10

5. Integración con la interfaz gráfica **10**

6. Diagrama de clases **14**

7. Pruebas **15**
7.1. Prueba 1: Gramática 1 (expresiones aritméticas) . . . 15
7.2. Prueba 2: Gramática 2 ($D \to D\ D$) . . . 16
7.3. Prueba 3: Gramática 3 (declaraciones con $T \to \varepsilon$) . . . 17
7.4. Prueba 4: Gramática 4 (sentencias) . . . 17
7.5. Prueba 5: Gramática 5 (arreglos) . . . 18
7.6. Prueba 6: Gramática 6 (expresiones LL(1)) . . . 19

Distribución de responsabilidades **20**

Conclusiones **21**

Recursos **22**

Referencias **22**

Introducción

En las primeras entregas el equipo construyó el analizador léxico del compilador de C++ a Java, primero como un Autómata Finito Determinista programado a mano (Asignación 1) y después generado con Flex (Asignación 2). En la Asignación 3 el proyecto entró a la fase de **análisis sintáctico** con el cálculo de los conjuntos PRIMERO y SIGUIENTE. Este informe documenta la cuarta entrega (Asignación 4), que continúa con el Tema 3 del curso.

Los analizadores sintácticos ascendentes de la familia LR deciden en cada paso si desplazan el siguiente token o si reducen por una producción. Para tomar esa decisión usan un autómata finito determinista cuyos estados indican qué parte de cada producción ya se reconoció. Cada estado es un conjunto de **elementos LR(0)**, y el conjunto de todos los estados es la **colección canónica** de la gramática. Con ella se construye después la tabla de análisis del analizador.

El objetivo de la asignación es implementar el algoritmo de la colección canónica junto con sus dos funciones auxiliares, **cerradura** e **Ir_a**, para calcular los estados del AFD a partir de una gramática libre de contexto aumentada leída de un archivo [6]. El programa muestra en una ventana la gramática de entrada, la gramática aumentada y todos los estados con sus conjuntos de elementos. El cálculo se integró al mismo proyecto de las entregas anteriores, como una ventana hija modal que se abre desde el menú *Analizadores* → *Analizador sintáctico*.

El trabajo se organizó en siete partes: (1) análisis de los conceptos y del formato de entrada; (2) estudio de los algoritmos de cerradura, Ir_a y colección canónica; (3) descripción de cómo se implementaron; (4) implementación en Python con la arquitectura por capas; (5) integración con la interfaz gráfica; (6) pruebas con las seis gramáticas de la asignación; y (7) distribución de responsabilidades entre los seis integrantes del equipo.

1 Análisis

1.1 Análisis sintáctico ascendente y gramática aumentada

Un analizador sintáctico ascendente construye el árbol de derivación desde las hojas hacia la raíz. Lee la tira de tokens de izquierda a derecha y, en cada paso, *desplaza* un token a la pila o *reduce* los símbolos del tope de la pila que coinciden con el lado derecho de una producción [1]. El método LR, propuesto por Knuth [3], guía esas decisiones con un autómata, y su variante más sencilla, SLR, fue propuesta por DeRemer [4].

El autómata se construye sobre la **gramática aumentada** G'. Es la misma gramática $G = (N, T, P, S)$ [5], con un símbolo inicial nuevo S' y una producción inicial nueva. En el curso esa producción incluye el marcador de fin de cadena [6, 7]:

$$S' \to S\,\$$$

Así, el analizador acepta la entrada justo cuando, después de reconocer una S completa, encuentra el \$.

1.2 Elementos LR(0), cerradura e Ir_a

Según Aho *et al.* [1, sección 4.6.2]:

- Un **elemento LR(0)** es una producción con un punto en alguna posición de su lado derecho. La producción $A \to XYZ$ da cuatro elementos: $[A \to \cdot XYZ]$, $[A \to X \cdot YZ]$, $[A \to XY \cdot Z]$ y $[A \to XYZ\cdot]$. La producción $A \to \varepsilon$ da un solo elemento, $[A \to \cdot]$. El punto separa lo que ya se reconoció (a su izquierda) de lo que falta por reconocer (a su derecha).
- La **cerradura** de un conjunto de elementos I agrega, por cada elemento $[A \to \alpha \cdot B\beta]$ con B no terminal, los elementos $[B \to \cdot\gamma]$ de todas las producciones de B. Si se espera ver una B, también puede empezar cualquiera de sus producciones.
- Ir_a(I, X) es la cerradura de todos los elementos $[A \to \alpha X \cdot \beta]$ tales que $[A \to \alpha \cdot X\beta]$ está en I. Es decir, avanza el punto sobre X. Es la transición del AFD desde el estado I con el símbolo X.
- La **colección canónica** es el conjunto de estados $C = \{I_0, I_1, \ldots\}$ que se alcanzan desde $I_0 =$ cerradura$(\{[S' \to \cdot S\,\$]\})$ aplicando Ir_a con todos los símbolos gramaticales.

Por ejemplo, en la Gramática 1, I_0 contiene $[E' \to \cdot E\$]$. Como después del punto hay una E, la cerradura agrega $[E \to \cdot E + T]$ y $[E \to \cdot T]$; por la T agrega $[T \to \cdot T * F]$ y $[T \to \cdot F]$, y por la F agrega $[F \to \cdot(E)]$ y $[F \to \cdot\text{id}]$. En cambio, Ir_a(I_0, F) solo contiene $[T \to F\cdot]$, un elemento *completo*: ya se reconoció todo el lado derecho de $T \to F$.

1.3 Herramientas utilizadas

Tabla 1: Herramientas utilizadas en la Asignación 4.

Herramienta	Uso
Python 3	Lenguaje del lector de gramáticas y del cálculo de la colección canónica.
Tkinter	Interfaz gráfica: ventana hija modal, diálogos para abrir y guardar archivos y mensajes de error.
Visual Studio Code	Edición del código y de las gramáticas aumentadas de prueba.
LaTeX	Elaboración de este informe.

1.4 Formato del archivo de la gramática aumentada

El enunciado pide como entrada un archivo con una gramática libre de contexto aumentada, con sus no terminales, terminales, símbolo inicial y reglas de producción [6]. Se reutilizó el formato de la Asignación 3 y se le agregó la producción aumentada como primera regla, igual que en el ejemplo del enunciado:

```
E T F            <- línea 1: no terminales
+ * ( ) id       <- línea 2: terminales
E'→E$             <- primera regla: la producción aumentada S' → S $
E→E+T             <- después, las reglas de la gramática original
E→T
...
```

- Como en el ejemplo del enunciado, E' y $ **no** tienen que declararse en el encabezado. Si la primera regla termina en $ y su lado izquierdo no está declarado, se toma como la producción aumentada: su lado izquierdo pasa a ser el símbolo inicial y $ se agrega a los terminales.
- Si el archivo no trae la producción aumentada, el programa la agrega: $S' \to S\,\$$. Si S' ya existe en la gramática (como E' en la Gramática 6), se usa S'', y así sucesivamente.
- Se conservan las reglas de la Asignación 3: la flecha puede ser →, `->` o `::=`; la cadena vacía se escribe ε o `eps`; se permiten alternativas con `|`; y los símbolos pueden ir juntos o separados por espacios.

Las seis gramáticas de la asignación, ya aumentadas, se guardaron en la carpeta nueva `gramaticasAumentadas`.

2 Algoritmos de la colección canónica

Los tres algoritmos se tomaron del enunciado de la asignación [6] y coinciden con los de Aho *et al.* [1, sección 4.6.2] y Louden [2, capítulo 5].

2.1 Algoritmo de cerradura

```
cerradura(I)
hacer
    para cada elemento [A → α·Bβ] en I, con B no terminal
        para cada producción B → γ
            si [B → ·γ] no está en I
                añadir [B → ·γ] a I
mientras haya elementos por añadir a I
return I
```

2.2 Algoritmo Ir_a

```
Ir_a(I, X)
J := {}                                   // conjunto de elementos vacío
para cada elemento [A → α·Xβ] que esté en I
    agregar el elemento [A → αX·β] a J
return cerradura(J)
```

Al conjunto J, antes de aplicarle la cerradura, se le llama **núcleo** del nuevo estado. La salida del programa lo muestra entre paréntesis, por ejemplo `cerr(Ir_a(I0,E))=cerr({[E'→E·$][E→E·+T]})`.

2.3 Algoritmo de la colección canónica

```
coleccionCanonica(G')
C := {I0 := cerradura({[S' → ·S$]})}
hacer
    para cada conjunto de elementos Ii en C
        para cada símbolo gramatical ·X de cada elemento de Ii
            si X es $ entonces
                Ir_a(Ii, X) := aceptación
            sino si Ir_a(Ii, X) no está vacío y no está en C entonces
                añadir a C: Ii+1 := Ir_a(Ii, X), como un nuevo estado
mientras haya más conjuntos de elementos a añadir a C
```

El símbolo \$ solo aparece después del punto en el elemento $[S' \to S \cdot \$]$. Avanzar sobre él significa que ya se reconoció toda la entrada, por eso esa transición se marca como **aceptación** en lugar de crear un estado.

2.4 Terminación del algoritmo

Una gramática tiene un número finito de elementos: cada producción $A \to \alpha$ da $|\alpha|+1$ elementos. Por lo tanto también hay un número finito de conjuntos de elementos posibles. La cerradura termina porque solo agrega elementos que todavía no están en I. La colección canónica termina porque solo agrega estados que todavía no están en C. Tarde o temprano todos los Ir_a dan estados que ya existen, y en ese momento C está completa.

2.5 Ejemplo paso a paso: Gramática 1

La Tabla 2 muestra los doce estados de la Gramática 1 en el orden en que el algoritmo los descubre. Los datos salen de ejecutar el programa y coinciden con la salida del ejemplo de la asignación [6].

Tabla 2: Estados de la colección canónica de la Gramática 1.

Estado	Se obtiene de	Conjunto de elementos
I_0	cerradura inicial	[E'→·E\$] [E→·E+T] [E→·T] [T→·T*F] [T→·F] [F→·(E)] [F→·id]
I_1	Ir_a(I_0, E)	[E'→E·\$] [E→E·+T]
I_2	Ir_a(I_0, T)	[E→T·] [T→T·*F]
I_3	Ir_a(I_0, F)	[T→F·]
I_4	Ir_a(I_0, `(`)	[F→(·E)] [E→·E+T] [E→·T] [T→·T*F] [T→·F] [F→·(E)] [F→·id]
I_5	Ir_a(I_0, id)	[F→id·]
I_6	Ir_a(I_1, `+`)	[E→E+·T] [T→·T*F] [T→·F] [F→·(E)] [F→·id]
I_7	Ir_a(I_2, `*`)	[T→T*·F] [F→·(E)] [F→·id]
I_8	Ir_a(I_4, E)	[F→(E·)] [E→E·+T]
I_9	Ir_a(I_6, T)	[E→E+T·] [T→T·*F]
I_{10}	Ir_a(I_7, F)	[T→T*F·]
I_{11}	Ir_a(I_8, `)`)	[F→(E)·]

En total se calculan 22 Ir_a. Once crean un estado nuevo y los otros once llevan a un estado que ya existía, por ejemplo Ir_a$(I_4, T) = I_2$ o Ir_a$(I_6, \texttt{(}) = I_4$. Además, Ir_a$(I_1, \$)$ es la aceptación.

La Figura 1 muestra cómo se descubren los estados: cada flecha es el Ir_a que creó el estado al que apunta.

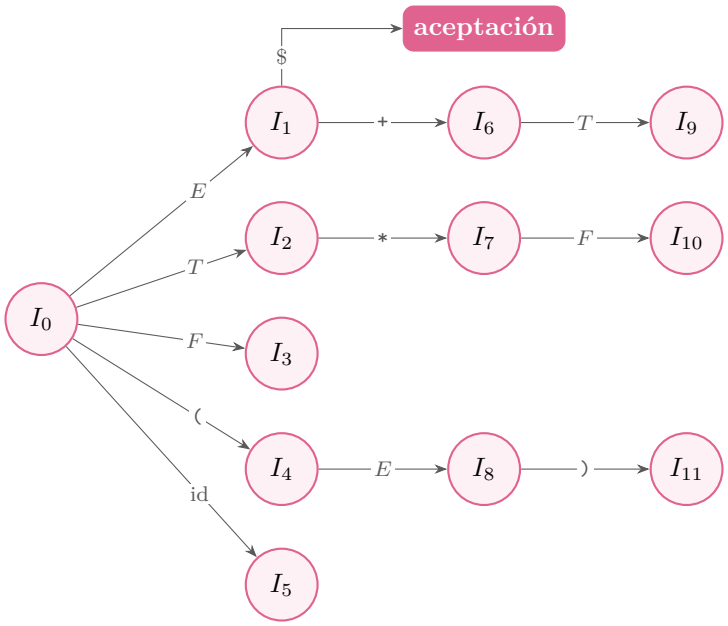


Figura 1: Orden en que se descubren los estados de la Gramática 1. Solo se dibujan los Ir_a que crean un estado nuevo; los que llevan a un estado existente aparecen en la salida del programa (Figura 7).

3 Descripción de la implementación

El cálculo de la colección canónica pasa por cuatro fases, cada una a cargo de una clase o función del paquete `persistencia`.

1. Lectura y validación (`LectorGramatica`). Se reutilizó el lector de la Asignación 3, que separa el encabezado de las reglas, divide cada lado derecho en símbolos por *coincidencia más larga* y valida la gramática. Se le agregó el reconocimiento de la producción aumentada: si la primera regla termina en $ y su lado izquierdo no está declarado, ese lado izquierdo se agrega a los no terminales como símbolo inicial, y $ se agrega a los terminales.

2. Gramática aumentada (`aumentar()`). Si la gramática ya trae $S' \to S\,\$$, la función la deja como producción 0. Si no la trae, crea un símbolo S' que no choque con los existentes y agrega la producción al principio. Así los demás algoritmos siempre pueden suponer que la producción 0 es la aumentada.

3. Cerradura, Ir_a y colección canónica (`ColeccionCanonica`). La clase implementa los tres algoritmos de la sección 2. El método `calcular()` recorre la lista de estados mientras crece, y por cada estado calcula Ir_a con cada símbolo que aparece después de un punto. Además de los estados, guarda cada paso de la corrida (cerradura inicial, Ir_a que crea un estado, Ir_a que lleva a un estado existente y aceptación) para poder mostrarlos después.

4. Formato de la salida (`lineas()` y `reporte()`). Cada paso se escribe con la notación del curso: `cerradura({...})=` para el estado inicial, `cerr(Ir_a(Ii,X))=cerr({...})` para cada transición, seguido del nuevo estado `Ij={...}` o de `= Ij` si el estado ya existía, y `Aceptación` para la transición con \$. Cada elemento se escribe como `[A`$\to\alpha\cdot\beta$`]`.

4 Implementación en Python

4.1 Arquitectura por capas y paquetes

El proyecto sigue el *Ejemplo de la Arquitectura de Software y uso de GUI en Eclipse* del profesor [8], con los paquetes de Java convertidos a paquetes de Python. La capa de **persistencia** contiene las clases que leen la gramática y calculan la colección canónica, y la capa de **presentación** contiene las ventanas. Como en el proyecto del curso, el paquete del algoritmo se llama `ccanonica`. Los paquetes de las entregas anteriores se conservaron. La Tabla 3 muestra todos los paquetes del proyecto.

Tabla 3: Paquetes de `src` y su contenido. Los marcados con ⋆ son nuevos en la Asignación 4 y los marcados con ∘ se modificaron.

Paquete	Contenido
`compi.persistencia.analizadorLexico.alexico`	`afd_lexico.py`, `analizador_lexico.py` (Asignación 1)
`compi.persistencia.analizadorLexico.flex`	`compilador_flex.py` (Asignación 2)
`compi.persistencia.analizadorLexico.generales`	`tokens.py`, `lector_archivo.py`
`compi.persistencia.analizadorSintacticoLR.ccanonica` ⋆	`coleccion_canonica.py`: `ColeccionCanonica` y la función `aumentar()`
`compi.persistencia.analizadorSintacticoLR.generales` ∘	`gramatica.py`: `Gramatica`, `LectorGramatica`, `ErrorGramatica`. Ahora reconoce la producción aumentada.
`compi.persistencia.analizadorSintacticoLR.primerosYsiguientes`	`primeros_siguientes.py`: `PrimerosSiguientes` (Asignación 3)
`compi.presentacion` ∘	`compilador.py` (`main`) y `ventana_principal.py` (menús y logo), con la opción nueva del menú
`compi.presentacion.alexico`	Pantallas y subventanas del analizador léxico (AFD y Flex), estilos e iconos de la interfaz
`compi.presentacion.asintactico` ∘	`ventana_primeros_siguientes.py` y, nueva, `ventana_coleccion_canonica.py`: `VentanaColeccionCanonica`

Además de `src`, la carpeta del proyecto contiene los recursos que usan los analizadores:

```
Arquitectura_software_CC/
|-- src/                    paquetes compi.persistencia y compi.presentacion
|-- codigosFuentes/         programas .cpp de prueba del analizador léxico
|-- gramaticas/             gramáticas g1 ... g6 (.txt) de la Asignación 3
|-- gramaticasAumentadas/   gramáticas g1 ... g6 aumentadas (Asignación 4)
|-- flex/                   lexer.l del analizador en Flex
|-- imagenes/               logo_utm.png
|-- herramientas/           win_flex y TinyCC (para compilar lexer.l en Windows)
|-- README.md               instrucciones de uso
`-- iniciar.sh              inicia la interfaz en Linux y macOS
```

El proyecto se entrega empaquetado en el archivo `Arquitectura_software_CC.zip`, con la estructura anterior. No incluye los archivos que se generan al usarlo: las carpetas `__pycache__` de Python y los archivos que Flex crea al compilar y ejecutar `lexer.l`.

4.2 Representación de los elementos y los estados

Un elemento $[A \to \alpha \cdot \beta]$ se representa con la tupla `(p, k)`, donde `p` es el índice de la producción $A \to \alpha\beta$ en la gramática aumentada y `k` es la posición del punto. Por ejemplo, en la Gramática 1 el elemento $[E \to E + \cdot T]$ es `(1, 2)`: producción 1 ($E \to E + T$) con el punto después de dos símbolos. El elemento $[A \to \cdot]$ de una producción vacía es `(p, 0)`, y es completo desde el inicio.

Un estado es una *lista* de elementos, para conservar el orden en que la cerradura los agrega y mostrarlos igual que en el ejemplo. Para saber si un estado ya existe, se compara como `frozenset`, porque dos estados son iguales si tienen los mismos elementos, sin importar el orden.

El símbolo que está después del punto se obtiene así; si el punto está al final, el elemento es completo y no hay símbolo:

```python
def simbolo_despues_punto(self, elemento):
    p, k = elemento
    derecha = self.gramatica.producciones[p][1]
    return derecha[k] if k < len(derecha) else None
```

4.3 Gramática aumentada

```python
def aumentar(gramatica):
    g = gramatica
    del_inicial = [p for p in g.producciones if p[0] == g.inicial]
    if (len(del_inicial) == 1 and len(del_inicial[0][1]) == 2
            and del_inicial[0][1][1] == FIN
            and g.es_no_terminal(del_inicial[0][1][0])):
        resto = [p for p in g.producciones if p[0] != g.inicial]
        return Gramatica(g.no_terminales, g.terminales, g.inicial,
                         del_inicial + resto)        # ya viene aumentada

    nuevo = g.inicial + "'"                      # S' no debe chocar con otro símbolo
    while nuevo in g.no_terminales or nuevo in g.terminales:
        nuevo += "'"
    terminales = g.terminales + ([FIN] if FIN not in g.terminales else [])
    return Gramatica([nuevo] + g.no_terminales, terminales, nuevo,
                     [(nuevo, (g.inicial, FIN))] + g.producciones)
```

4.4 Cerradura

La lista `resultado` se recorre mientras crece: cada elemento que se agrega también se revisa más adelante. Eso equivale al ciclo *hacer ... mientras haya elementos por añadir* del pseudocódigo, sin tener que repetir pasadas completas.

```
def cerradura(self, elementos):
    resultado = list(elementos)
    i = 0
    while i < len(resultado):
        b = self.simbolo_despues_punto(resultado[i])
        if b is not None and self.gramatica.es_no_terminal(b):  # [A → α·Bβ]
            for p, (izq, _) in enumerate(self.gramatica.producciones):
                if izq == b and (p, 0) not in resultado:          # [B → ·γ]
                    resultado.append((p, 0))
        i += 1
    return resultado
```

4.5 Ir_a

```
def nucleo_ir_a(self, estado, x):
    """J = {[A → αX·β] | [A → α·Xβ] en el estado}."""
    return [(p, k + 1) for p, k in estado
            if self.simbolo_despues_punto((p, k)) == x]

def ir_a(self, estado, x):
    return self.cerradura(self.nucleo_ir_a(estado, x))
```

4.6 Colección canónica

```
def calcular(self):
    self.estados = [self.cerradura([(0, 0)])]             # I0
    self.transiciones = {}
    self.pasos = [("inicio", [(0, 0)])]
    conjuntos = [frozenset(self.estados[0])]
    i = 0
    while i < len(self.estados):                          # C crece mientras se recorre
        for x in self.simbolos_transicion(self.estados[i]):
            if x == FIN:                                  # Ir_a(Ii, $) = aceptación
                self.transiciones[(i, x)] = ACEPTACION
                self.pasos.append(("aceptacion", i))
                continue
            nucleo = self.nucleo_ir_a(self.estados[i], x)
            destino = self.cerradura(nucleo)
            clave = frozenset(destino)
            nuevo = clave not in conjuntos
            if nuevo:                                     # no está en C: nuevo estado
                self.estados.append(destino)
                conjuntos.append(clave)
            j = conjuntos.index(clave)
            self.transiciones[(i, x)] = j
            self.pasos.append(("ir_a", i, x, nucleo, j, nuevo))
        i += 1
    return self
```

El método `simbolos_transicion()` devuelve los símbolos que están después de un punto en el estado, sin repetir y en el orden en que aparecen. Por eso los estados se numeran en el mismo orden que en el ejemplo de la asignación: desde I_0 primero se visita E, luego T, F, `(` e id.

4.7 Manejo de errores en la gramática

Los errores de formato se detectan con el mismo `LectorGramatica` de la Asignación 3: archivo vacío, falta de reglas, símbolos que son terminal y no terminal a la vez, lados izquierdos o símbolos del lado derecho no declarados y no terminales sin reglas. En todos los casos se lanza `ErrorGramatica` con un mensaje que indica la línea del problema, y la ventana lo muestra en un cuadro de diálogo sin cerrar el programa (Figura 2).

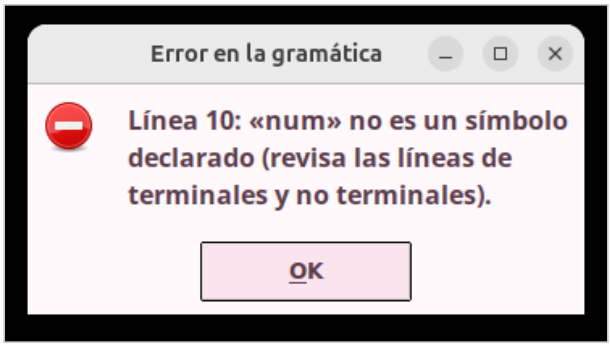


Figura 2: Error al analizar la Gramática 1 aumentada con una regla extra `F→num`: `num` no está declarado como terminal.

4.8 Relación con las entregas anteriores

La colección canónica reutiliza la representación de la gramática de la Asignación 3: la clase `Gramatica`, el lector y la representación de $A \to \varepsilon$ como un lado derecho vacío. Gracias a esa representación, la producción vacía da un único elemento completo `(p, 0)` sin código especial.

El algoritmo no necesita los conjuntos PRIMERO y SIGUIENTE ni el analizador léxico, porque su entrada es una gramática y no un programa en C++. Como en la entrega anterior, la nueva ventana reutiliza los colores, las fuentes y los iconos de la interfaz para que todo el compilador tenga la misma apariencia.

5 Integración con la interfaz gráfica

Al iniciar, la interfaz muestra el logo de la Universidad como pantalla principal (Figura 3). Al submenú *Analizador sintáctico* se le agregó la opción *Algoritmo Colección Canónica*, debajo de *Algoritmo Primeros y Siguientes*.

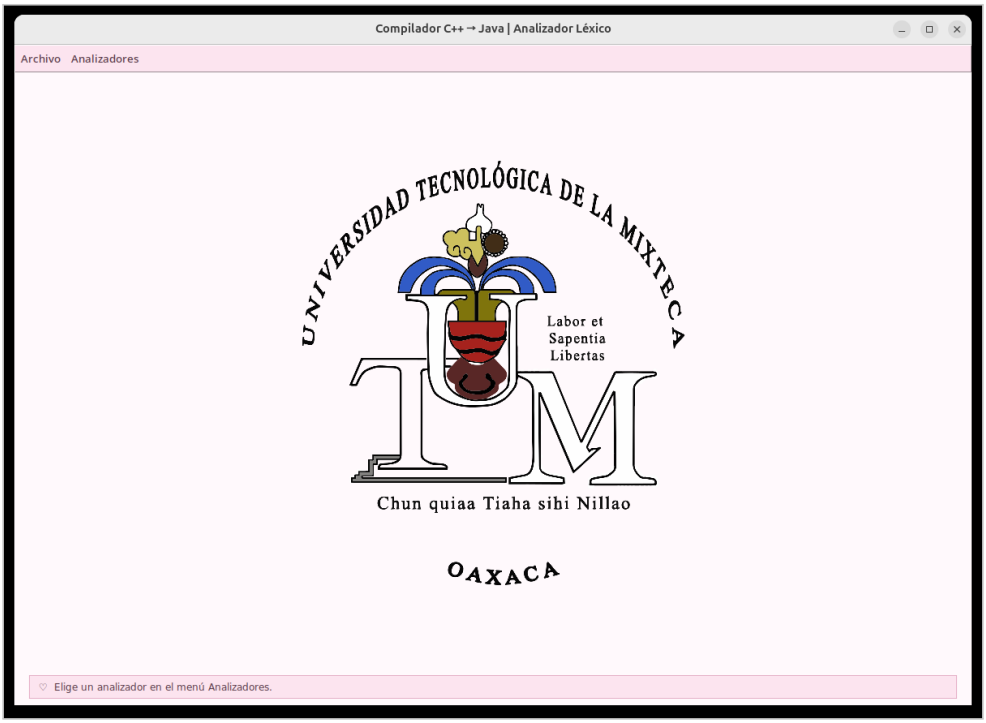


Figura 3: Pantalla principal del compilador. El cálculo se abre desde *Analizadores* → *Analizador sintáctico* → *Algoritmo Colección Canónica*.

Igual que en la entrega anterior, la ventana principal abre el algoritmo como una ventana hija modal, equivalente al `JDialog` del ejemplo del profesor. La clase `VentanaColeccionCanonica` hereda de `tk.Toplevel` [9]:

```
# VentanaPrincipal
mn_asintactico.add_command(label="Algoritmo Colección Canónica",
                           command=self.coleccion_canonica)

def coleccion_canonica(self):
    VentanaColeccionCanonica(self, True)          # ventana hija modal

# VentanaColeccionCanonica(tk.Toplevel)
def __init__(self, padre, modal):
    super().__init__(padre.root)
    self.padre = padre
    self.lector = LectorGramatica()
    self.resultado = None
    self.ini()
    if modal:
        self.grab_set()
```

La ventana tiene cuatro zonas (Figura 4):

- **Formulario:** botón *Abrir* para elegir un archivo de la carpeta `gramaticasAumentadas`, la ruta seleccionada y los botones *Analizar*, *Limpiar* y *Guardar resultados*.
- **Gramática de entrada:** el archivo, editable, con una ayuda sobre el formato.
- **Gramática aumentada:** la gramática con la que se calculó la colección, con la producción

aumentada en primer lugar.

- **Colección canónica:** la cerradura inicial, cada Ir_a y cada estado. Para facilitar la lectura, los estados nuevos aparecen en rosa, los Ir_a que llevan a un estado existente en gris y la aceptación en verde. Debajo hay una barra de estado con el número de estados calculados.



Figura 4: Ventana *Cálculo de la colección canónica* recién abierta. *Analizar* y *Guardar resultados* siguen desactivados porque todavía no hay gramática.

Los botones hacen lo siguiente:

- **Abrir** (`abrir_archivo()`): muestra un `filedialog` que empieza en la carpeta `gramaticasAumentadas` y copia el archivo al panel de la gramática de entrada (Figura 5).
- **Analizar** (`analizar()`): toma el texto *actual* del panel, lo pasa a `LectorGramatica.leer()`, calcula `ColeccionCanonica(gramatica).calcular()` y muestra la gramática aumentada y la colección.
- **Guardar resultados** (`guardar_resultados()`): escribe en un `.txt` la gramática aumentada y la salida de `reporte()`.
- **Limpiar** (`limpiar()`): vacía todos los paneles.



Figura 5: Gramática 1 aumentada (`g1_expresiones_aumentada.txt`) cargada con el botón *Abrir*, antes de analizarla.

Como en la entrega anterior, todo el cálculo está en la capa de persistencia, que no importa nada de `tkinter`, así que el algoritmo también se puede ejecutar sin interfaz. Para iniciar la interfaz, desde la carpeta `src`:

```
$ python3 -m compi.presentacion.compilador
```

6 Diagrama de clases

Las clases de esta entrega se reparten en las dos capas de la arquitectura. La Figura 6 muestra sus atributos, sus métodos principales y cómo se relacionan. `LectorGramatica`, `Gramatica` y `ErrorGramatica` vienen de la Asignación 3.

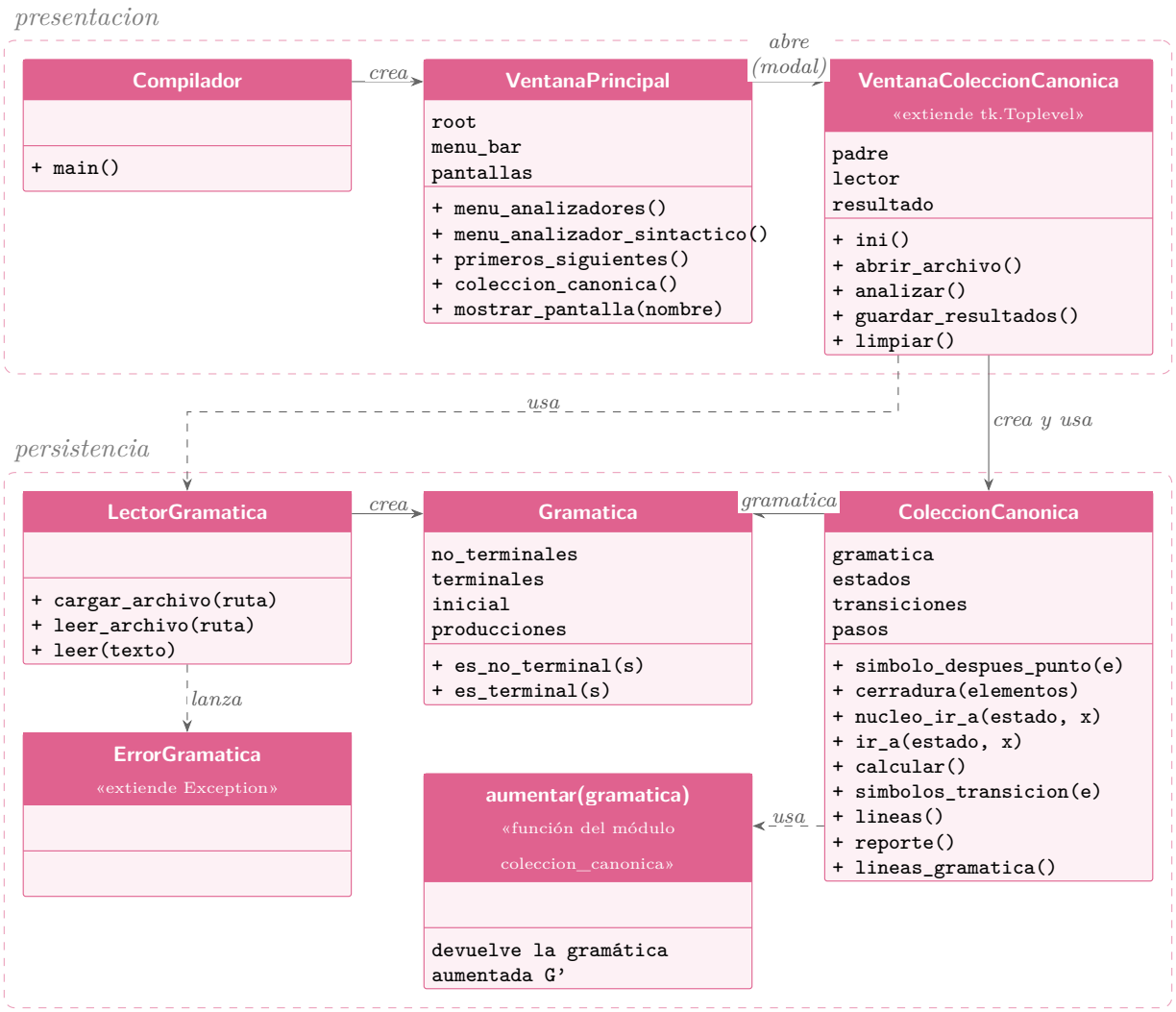


Figura 6: Diagrama de clases del cálculo de la colección canónica. Flecha continua: asociación o creación; flecha punteada: dependencia.

7 Pruebas

Las pruebas se ejecutaron desde la interfaz (*Abrir* → *Analizar*) con las seis gramáticas de la asignación, ya aumentadas y guardadas en la carpeta `gramaticasAumentadas`. La salida de la Gramática 1 se comparó con el ejemplo de la asignación, y la de las demás con el cálculo manual de sus estados. La Tabla 4 resume lo que prueba cada gramática y la Tabla 5 los resultados del programa. Las figuras muestran la salida real.

Tabla 4: Gramáticas aumentadas de prueba y lo que ejercita cada una.

Gram.	Archivo	Prod. de G'	Caso que prueba
1	`g1_expresiones_aumentada.txt`	7	Ejemplo de la asignación; E' y \$ sin declarar en el encabezado.
2	`g2_declaraciones_aumentada.tx`	7	No terminal repetido ($D \to D\,D$) y elemento vacío [$L \to \cdot$].
3	`g3_declaraciones_aumentada.tx`	8	Producciones vacías desde el estado inicial ($T \to \varepsilon$).
4	`g4_sentencias_aumentada.txt`	10	La gramática más grande; terminal de varios caracteres (`:=`).
5	`g5_arreglos_aumentada.txt`	8	No terminal con apóstrofo (V') que deriva ε.
6	`g6_expresiones_ll1_aumentada.`	9	E' ya existe, así que el símbolo aumentado es E''.

Tabla 5: Resultados del programa para las seis gramáticas.

Gram.	Inicial de G'	Estados	Ir_a calculados	Estados nuevos	A estado existente
1	E'	12 (I_0–I_{11})	22	11	11
2	D'	12 (I_0–I_{11})	19	11	8
3	D'	12 (I_0–I_{11})	15	11	4
4	S'	23 (I_0–I_{22})	44	22	22
5	S'	14 (I_0–I_{13})	14	13	1
6	E''	16 (I_0–I_{15})	26	15	11

En las seis gramáticas la aceptación es Ir_a(I_1, \$), porque $I_1 = \text{Ir_a}(I_0, S)$ siempre es el primer estado que se descubre y contiene [$S' \to S \cdot \$$].

7.1 Prueba 1: Gramática 1 (expresiones aritméticas)

Es la gramática del ejemplo de la asignación. La salida coincide exactamente con la del enunciado: los mismos doce estados, con los mismos elementos y en el mismo orden (Tabla 2).



Figura 7: Resultado de la Gramática 1 (`g1_expresiones_aumentada.txt`).

7.2 Prueba 2: Gramática 2 ($D \to D\ D$)

Como D va seguida de otra D, el estado I_1 contiene $[D \to D \cdot D]$ y la cerradura vuelve a agregar todas las producciones de D y de T. Por la misma razón, $\text{Ir_a}(I_5, D) = I_5$: el estado tiene una transición a sí mismo. La regla $L \to \varepsilon$ da el elemento completo $[L \to \cdot]$, que aparece en I_6 y en I_{10} y no genera transiciones.



Figura 8: Resultado de la Gramática 2 (`g2_declaraciones_aumentada.txt`).

7.3 Prueba 3: Gramática 3 (declaraciones con $T \to \varepsilon$)

Por $T \to \varepsilon$, el estado inicial I_0 ya contiene el elemento completo $[T \to \cdot]$. La regla $L \to \mathrm{id}\, L$ hace que $\mathrm{Ir_a}(I_6, \mathrm{id}) = I_6$. Es la gramática con menos Ir_a que llevan a un estado existente después de la Gramática 5.



Figura 9: Resultado de la Gramática 3 (`g3_declaraciones_aumentada.txt`).

7.4 Prueba 4: Gramática 4 (sentencias)

Es la gramática más grande: 23 estados y 44 Ir_a. Las producciones $E \to E + E$ y $E \to (S, E)$ hacen que los estados que esperan una E (I_5, I_6, I_{14}, I_{17}, I_{19}) compartan sus transiciones con id, num y `(`, por eso la mitad de los Ir_a llevan a un estado existente. El terminal `:=` tiene dos caracteres y aparece como un solo símbolo en $[S \to \mathrm{id}\cdot := E]$ gracias a la coincidencia más larga del lector.



Figura 10: Resultado de la Gramática 4 (`g4_sentencias_aumentada.txt`).

7.5 Prueba 5: Gramática 5 (arreglos)

V' se lee como un solo símbolo y no como V seguido de un apóstrofo. Casi todos los estados son nuevos: solo Ir_a(I_{12}, [) = I_{10} regresa a un estado existente, que es el que permite declarar varias dimensiones `[nint][nint]...`



Figura 11: Resultado de la Gramática 5 (`g5_arreglos_aumentada.txt`).

7.6 Prueba 6: Gramática 6 (expresiones LL(1))

Esta gramática ya usa el no terminal E', por eso el símbolo inicial de la gramática aumentada es E'' y la producción aumentada es $E'' \to E\,\$$. Las alternativas con | se separan en producciones independientes, y las dos producciones vacías dan los elementos completos $[E' \to \cdot]$ y $[T' \to \cdot]$.



Figura 12: Resultado de la Gramática 6 (`g6_expresiones_ll1_aumentada.txt`).

En las seis gramáticas, los estados calculados por el programa coincidieron con el ejemplo de la asignación (Gramática 1) y con el cálculo manual (Gramáticas 2 a 6).

Distribución de responsabilidades

Tabla 6: Módulos del cálculo de la colección canónica, funciones y responsables.

Módulo	Clase y funciones	Responsable	Actividad
Estructura y menú	`VentanaPrincipal`: `coleccion_canonica()`; paquete `ccanonica`; carpeta `gramaticasAumentadas`	Arenas Rojas I. N.	Organiza los paquetes nuevos, agrega la opción *Algoritmo Colección Canónica* al menú y abre la ventana hija modal. Coordina la planificación y el informe.
Interfaz gráfica	`VentanaColeccionCanonica`: `ini()`, `abrir_archivo()`, `analizar()`, `limpiar()`, `guardar_resultados()`	Bautista Jiménez I.	Construye la ventana con los botones, los paneles de la gramática de entrada, la gramática aumentada y la colección, y la barra de estado.
Gramática aumentada	`aumentar()`; cambios a `LectorGramatica`	Hernández Torija I. A.	Reconoce la producción aumentada en el archivo y la agrega cuando falta, evitando choques de nombres (E''). Elabora el diagrama de clases.
Cerradura e Ir_a	`simbolo_despues_punto()`, `cerradura()`, `nucleo_ir_a()`, `ir_a()`	Juárez Jiménez Y. S.	Representa los elementos como `(p, k)` e implementa la cerradura de un conjunto y el avance del punto sobre un símbolo.
Colección canónica y salida	`calcular()`, `simbolos_transicion()`, `formatear_elemento()`, `lineas()`, `reporte()`	López López X. N.	Calcula los estados del AFD, detecta los estados repetidos y la aceptación, y da formato a la salida con la notación del curso.
Pruebas	`gramaticasAumentadas/g1` a `g6`	Quiroz Martínez M.	Prepara las seis gramáticas aumentadas, ejecuta las pruebas y las compara con el ejemplo y con el cálculo manual.

Tabla 7: Cronograma de la Asignación 4 (12 al 16 de octubre de 2026).

Fecha	Fase	Actividades	Responsables
12/10	Análisis y reparto	Requerimientos, estudio de los elementos LR(0) y de los algoritmos de cerradura, Ir_a y colección canónica, y reparto de módulos.	Todo el equipo
13/10	Diseño	Representación de los elementos y los estados, formato de la gramática aumentada, paquete `ccanonica` y gramáticas aumentadas de prueba.	Arenas, Hernández, Juárez, Quiroz
14/10	Implementación	Gramática aumentada, cerradura, Ir_a, colección canónica, formato de la salida, ventana y opción del menú.	Arenas, Bautista, Hernández, Juárez, López
15/10	Integración y pruebas	Conexión de la ventana con el lector y el cálculo, pruebas con las seis gramáticas, diagrama de clases e informe.	Todo el equipo
16/10	Revisión y entrega	Reunión final, ensamblado del informe, empaquetado del proyecto y entrega.	Todo el equipo

Conclusiones

El algoritmo de la colección canónica se implementó tal como lo plantean el enunciado y la bibliografía del curso: la cerradura agrega los elementos de las producciones que se esperan ver, Ir_a avanza el punto sobre un símbolo, y la colección se construye aplicando Ir_a a cada estado hasta que no aparecen estados nuevos. La salida de la Gramática 1 coincidió con el ejemplo de la asignación, estado por estado y en el mismo orden, y las otras cinco gramáticas coincidieron con el cálculo manual.

Tres decisiones de diseño simplificaron el código. La primera fue representar cada elemento como una tupla `(p, k)`, que hace inmediato avanzar el punto y saber si un elemento está completo. La segunda fue guardar cada estado como lista, para conservar el orden de la salida, y compararlo como conjunto, para detectar los estados repetidos. La tercera fue reutilizar la representación de la Asignación 3, donde $A \to \varepsilon$ es un lado derecho vacío: así la producción vacía da un único elemento completo sin casos especiales.

La arquitectura por capas permitió agregar el algoritmo sin cambiar las entregas anteriores: el paquete `ccanonica` reutiliza el lector de gramáticas y la nueva ventana solo se conectó al menú existente. Los estados y las transiciones que calcula este algoritmo son la base para construir, en la siguiente entrega, la tabla de análisis sintáctico del analizador.

Recursos

Enlace de Google Drive con las actividades realizadas por el equipo: *informeActividadesEq3*.

Referencias

[1] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). *Compiladores: principios, técnicas y herramientas* (2.ª ed.). Pearson Educación.

[2] Louden, K. C. (2004). *Construcción de compiladores: principios y práctica*. Thomson.

[3] Knuth, D. E. (1965). On the translation of languages from left to right. *Information and Control*, 8(6), 607–639. https://doi.org/10.1016/S0019-9958(65)90426-2

[4] DeRemer, F. L. (1971). Simple LR(k) grammars. *Communications of the ACM*, 14(7), 453–460. https://doi.org/10.1145/362619.362625

[5] Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introducción a la teoría de autómatas, lenguajes y computación* (3.ª ed.). Pearson Educación.

[6] Martínez Torres, D. (2026). *Asignación 4: Implementación del algoritmo de la Colección canónica*. Curso de Compiladores, Instituto de Computación, Universidad Tecnológica de la Mixteca.

[7] Martínez Torres, D. *Tema 3. Análisis sintáctico* (presentación del curso). Universidad Tecnológica de la Mixteca.

[8] Martínez Torres, D. *Ejemplo de la Arquitectura de Software y uso de GUI en Eclipse*. Material del curso de Compiladores, Universidad Tecnológica de la Mixteca.

[9] Python Software Foundation. *tkinter — Python interface to Tcl/Tk*. Documentación de Python 3. https://docs.python.org/3/library/tkinter.html